

Local Run and Production Deployment Runbook

A practical operations guide for setting up, running, validating, deploying, monitoring, and troubleshooting the Agentic Research Agent project.

Document type: Operations Runbook

Generated: 2026-06-04

Project: agentic-ai-learning

Runtime: Python 3.13

What This Project Runs As

This project currently ships as a Python package with a Typer command-line interface named `research-agent`. It can also be used as a library by importing `ResearchAgent`. There is no HTTP server, Dockerfile, Kubernetes manifest, or managed API deployment in the repository yet.

Supported now

Local CLI usage, local chat, RAG ingestion, test/lint/type-check workflow, and package/library usage.

Production-ready pattern now

Run as a scheduled job, internal CLI tool, worker process, or library inside another service.

Needs extra implementation

Public API deployment, authentication, request rate limiting, container image, health endpoints, and durable conversation storage.

Repository Map

```
src/agentic_research_agent/  
  agents/      LangGraph graph, prompts, state, service facade  
  config/      Pydantic settings and environment variable loading  
  core/        LLM factory, logging, exception hierarchy  
  schemas/     Request and response contracts  
  tools/       Calculator, web search, knowledge base retrieval  
  
data/  
  knowledge_base/  Source documents used by RAG ingestion  
  vector_store/    Generated Chroma index, ignored by git  
  
docs/  
  architecture.html  
  operations-runbook.html  
  
tests/  
  Offline unit tests and graph tests
```

Local Prerequisites

Requirement	Why it is needed	Check command
Python 3.13+	The package declares <code>requires-python</code> = <code>">=3.13"</code> .	<code>python --version</code>
uv	Dependency and virtual environment management.	<code>uv --version</code>
LLM provider access	Required for real agent answers unless using local Ollama.	Provider console or local Ollama check.
Internet access	Needed for dependency install, first embedding model download, and web search.	<code>curl https://pypi.org</code>
Local disk space	Embedding model cache and Chroma vector store require storage.	<code>df -h</code>

Tip: The first RAG ingestion can take longer because the embedding model is downloaded and cached locally.

Local Setup From Fresh Clone

1. Enter the project directory

```
cd "/home/joginder-singh/Study/Agentic AI/langchain"
```

2. Install dependencies

```
uv sync
```

This creates or updates the local virtual environment and installs runtime plus development dependencies from `pyproject.toml` and `uv.lock`.

3. Create local environment file

```
cp .env.example .env
```

4. Configure one LLM provider

For Groq, edit `.env` and set:

```
LLM_PROVIDER=groq
LLM_MODEL=llama-3.3-70b-versatile
GROQ_API_KEY=your_real_key_here
```

For OpenAI:

```
LLM_PROVIDER=openai
LLM_MODEL=gpt-4o-mini
OPENAI_API_KEY=your_real_key_here
```

For Anthropic:

```
LLM_PROVIDER=anthropic
LLM_MODEL=claude-sonnet-4-6
ANTHROPIC_API_KEY=your_real_key_here
```

For local Ollama:

```
ollama serve  
ollama pull llama3.1
```

```
LLM_PROVIDER=ollama  
LLM_MODEL=llama3.1  
OLLAMA_BASE_URL=http://localhost:11434
```

Secret rule: Never commit `.env`. It is ignored by git and must stay local or come from a production secret manager.

Build the Knowledge Base

The agent can search local documents from `data/knowledge_base`. Supported source formats are Markdown, text, reStructuredText, and PDF.

Normal ingestion

```
uv run research-agent ingest
```

Force rebuild after changing documents

```
uv run research-agent ingest --force
```

Add new documents

```
cp your-document.md data/knowledge_base/  
uv run research-agent ingest --force
```

The generated vector index is written to `data/vector_store`. This folder is intentionally ignored by git because it is generated data.

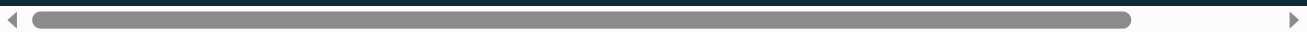
Run Locally

Ask one question

```
uv run research-agent ask "What is the ReAct pattern in agentic AI?"
```

Use a named conversation thread

```
uv run research-agent ask "Explain LangGraph memory in simple terms" --thread learning
```



Start interactive chat

```
uv run research-agent chat
```

Interactive chat with a custom thread

```
uv run research-agent chat --thread support-demo
```

Run as a Python library

```
uv run python - <<'PY'
from agentic_research_agent import ResearchAgent

agent = ResearchAgent()
response = agent.ask("Compute sqrt(144), then explain RAG in one sentence.")
print(response.answer)
print("run id:", response.run_id)
print("duration ms:", response.duration_ms)
print("tools:", [tool.name for tool in response.tool_calls])
PY
```

Developer Commands

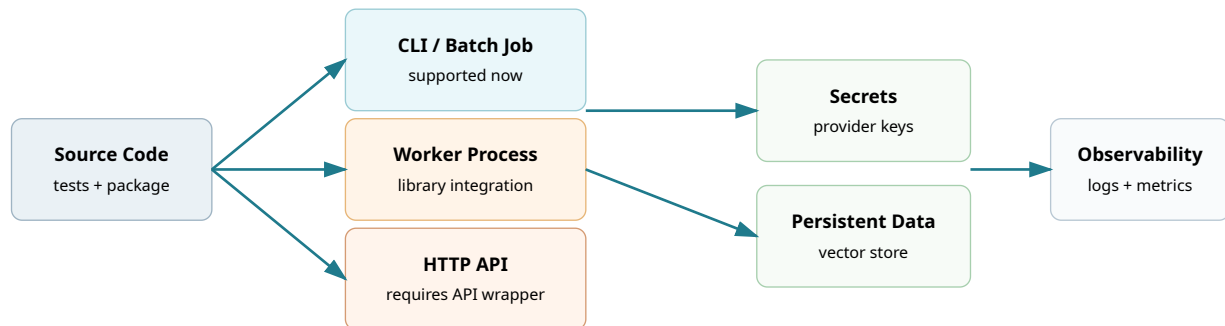
Command	Purpose	When to run
<code>uv sync</code>	Install or update dependencies.	After clone, dependency changes, or lockfile updates.
<code>uv run pytest</code>	Run automated tests.	Before committing or deploying.
<code>uv run ruff check .</code>	Run lint checks.	Before committing or deploying.
<code>uv run ruff format .</code>	Format Python code.	Before committing code changes.
<code>uv run mypy</code>	Run static type checking.	Before committing or deploying.
<code>uv build</code>	Build wheel and source distribution.	Before packaging or publishing.
<code>uv run research-agent ingest --force</code>	Rebuild RAG index.	After changing knowledge base documents or retrieval settings.

Environment Variables

Variable	Required	Default	Notes
ENVIRONMENT	No	development	Use production in production to enable production-oriented behavior such as JSON logs.
LOG_LEVEL	No	INFO	Allowed values: DEBUG, INFO, WARNING, ERROR, CRITICAL.
LLM_PROVIDER	No	groq	Allowed values: groq, anthropic, openai, ollama.
LLM_MODEL	No	provider-specific	Optional. Leave unset to use the selected provider's default (groq → llama-3.3-70b-versatile, openai → gpt-4o-mini, anthropic → claude-sonnet-4-6, ollama → llama3.1). Set to override; an explicit value must be valid for the chosen provider.
LLM_TEMPERATURE	No	0.1	Validated between 0.0 and 2.0.
LLM_MAX_TOKENS	No	1024	Maximum model response token budget.
LLM_TIMEOUT_SECONDS	No	60	Timeout passed into supported LLM providers.
GROQ_API_KEY	If Groq	None	Required when LLM_PROVIDER=groq.

Variable	Required	Default	Notes
<code>ANTHROPIC_API_KEY</code>	If Anthropic	None	Required when <code>LLM_PROVIDER=anthropic</code> .
<code>OPENAI_API_KEY</code>	If OpenAI	None	Required when <code>LLM_PROVIDER=openai</code> .
<code>OLLAMA_BASE_URL</code>	If custom Ollama	<code>http://localhost:11434</code>	Used when <code>LLM_PROVIDER=ollama</code> .
<code>EMBEDDING_MODEL</code>	No	<code>sentence-transformers/all-MiniLM-L6-v2</code>	Local embedding model for RAG ingestion and search.
<code>KNOWLEDGE_BASE_DIR</code>	No	<code>data/knowledge_base</code>	Directory containing source documents.
<code>VECTOR_STORE_DIR</code>	No	<code>data/vector_store</code>	Directory for generated Chroma index.
<code>CHUNK_SIZE</code>	No	<code>1000</code>	Must be greater than zero.
<code>CHUNK_OVERLAP</code>	No	<code>150</code>	Must be lower than <code>CHUNK_SIZE</code> .
<code>RETRIEVER_TOP_K</code>	No	<code>4</code>	Number of chunks returned by knowledge base search.
<code>MAX_SEARCH_RESULTS</code>	No	<code>5</code>	Maximum DuckDuckGo results returned to the agent.
<code>RECURSION_LIMIT</code>	No	<code>25</code>	LangGraph super-step limit for one run.

Production Deployment Options



The safest production path today is to deploy as a controlled CLI, batch job, or worker. An HTTP API should be added deliberately with authentication, health checks, and rate limiting.

Production Deployment: CLI or Batch Job

Use this pattern when the agent is run by trusted operators, cron, Airflow, Jenkins, GitHub Actions, or an internal automation worker.

1. Prepare host

```
python --version
uv --version
git --version
```

2. Fetch code and install

```
git clone <your-repository-url> agentic-research-agent
cd agentic-research-agent
uv sync --frozen
```

3. Provide production environment

```
export ENVIRONMENT=production
export LOG_LEVEL=INFO
export LLM_PROVIDER=groq
export LLM_MODEL=llama-3.3-70b-versatile
export GROQ_API_KEY="value-from-secret-manager"
export VECTOR_STORE_DIR="/var/lib/agentic-research-agent/vector_store"
export KNOWLEDGE_BASE_DIR="/opt/agentic-research-agent/knowledge_base"
```

4. Build or restore vector store

```
uv run research-agent ingest --force
```

For repeatable deployments, prefer building the vector store during release and restoring it as a versioned artifact or persistent volume.

5. Smoke test

```
uv run research-agent ask "Say hello and compute 12 * 12."
```

6. Schedule a job

```
# Example cron entry, runs daily at 08:00.  
0 8 * * * cd /opt/agentic-research-agent && /usr/local/bin/uv run research-agent ask
```

Production logging: Set `ENVIRONMENT=production`. The application emits JSON logs from the package logger, which are easier for log collectors to parse.

Production Deployment: Container Pattern

The repository does not currently include a Dockerfile. The following is a recommended pattern if you decide to containerize it.

Suggested Dockerfile

```
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    ENVIRONMENT=production

WORKDIR /app

RUN apt-get update \
    && apt-get install -y --no-install-recommends curl build-essential \
    && rm -rf /var/lib/apt/lists/*

RUN pip install --no-cache-dir uv

COPY pyproject.toml uv.lock README.md ./
COPY src ./src
COPY data/knowledge_base ./data/knowledge_base

RUN uv sync --frozen --no-dev

VOLUME ["/app/data/vector_store"]

ENTRYPOINT ["uv", "run", "research-agent"]
```

Build image

```
docker build -t agentic-research-agent:0.1.0 .
```

Run ingestion in container

```
docker run --rm \
    --env-file .env.production \
```

```
-v agentic_vector_store:/app/data/vector_store \  
agentic-research-agent:0.1.0 ingest --force
```

Ask a question in container

```
docker run --rm \  
  --env-file .env.production \  
  -v agentic_vector_store:/app/data/vector_store \  
  agentic-research-agent:0.1.0 ask "What is RAG?"
```

Important: Do not bake real provider keys into the image. Use runtime environment variables, Docker secrets, Kubernetes secrets, or a cloud secret manager.

Production Deployment: API Service Path

If the project must serve multiple users over HTTP, add a thin API layer around `ResearchAgent`. Do not expose the CLI process directly as a public service.

Minimum FastAPI shape

```
from fastapi import FastAPI
from pydantic import BaseModel
from agentic_research_agent import ResearchAgent

app = FastAPI()
agent = ResearchAgent()

class AskRequest(BaseModel):
    question: str
    thread_id: str = "default"

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/ask")
def ask(request: AskRequest):
    return agent.ask(request.question, thread_id=request.thread_id).model_dump()
```

API production controls to add

- Authentication and authorization for every request.
- Rate limits per user, team, or API key.
- Request size limits and timeout settings.
- Durable LangGraph checkpointing instead of in-memory `MemorySaver`.
- Central tracing for LLM calls, tool calls, errors, and latency.
- Health endpoints that verify vector store readiness and provider connectivity.

CI/CD Checklist

Before deploying a build, run the same validation locally and in CI.

```
uv sync --frozen
uv run ruff check .
uv run mypy
uv run pytest
uv build
```

Gate	Required result	Why it matters
Tests	Pass	Protects graph wiring, settings behavior, calculator safety, and provider factory assumptions.
Lint	Pass	Keeps codebase consistent and catches common defects.
Type check	Pass	Protects contracts across service, schemas, config, and tools.
Build	Pass	Confirms the package can be distributed or installed cleanly.
Secrets scan	Pass	Prevents API keys from entering source control or artifacts.

Operational Monitoring

Signal	How to observe	Alert when
Run success/failure	Application logs around <code>agent_run_started</code> and <code>agent_run_finished</code> .	Failure rate rises above normal baseline.
Latency	<code>duration_ms</code> in <code>AgentResponse</code> and logs.	p95 duration breaches user-facing SLA.
Tool usage	Tool call count and tool names from response metadata.	Unexpected tool usage spikes or drops.
Provider errors	LLM provider exceptions and timeout logs.	Authentication failures, rate limits, or provider outages occur.
RAG readiness	Presence and health of <code>data/vector_store/chroma.sqlite3</code> .	Vector store missing, stale, or corrupt.
Search availability	Recoverable web search error messages and warning logs.	Search failures become frequent.

Backup and Restore

What to back up

- `data/knowledge_base`: source documents, the canonical RAG corpus.
- `data/vector_store`: generated Chroma index, useful for faster restore.
- Production configuration in secret/config manager, not plain text files.

Backup command example

```
tar -czf knowledge-base-backup-$(date +%Y%m%d).tgz data/knowledge_base
```

Restore command example

```
tar -xzf knowledge-base-backup-20260604.tgz  
uv run research-agent ingest --force
```

If the vector store is not backed up, it can be regenerated from the knowledge base documents.

Troubleshooting

Symptom	Likely cause	Fix
<code>GROQ_API_KEY</code> is not set	Selected provider is Groq but key is missing.	Set <code>GROQ_API_KEY</code> in <code>.env</code> or secret manager.
Knowledge base directory not found	<code>KNOWLEDGE_BASE_DIR</code> points to a missing path.	Create the directory or correct the environment variable.
No documents found	No supported files under knowledge base directory.	Add <code>.md</code> , <code>.txt</code> , <code>.rst</code> , or <code>.pdf</code> files and ingest again.
Slow first ingestion	Embedding model is downloading for the first time.	Wait for the first run or pre-warm model cache in deployment.
Web search temporarily unavailable	DuckDuckGo throttling, network issue, or upstream failure.	Retry later, reduce traffic, or add a production search provider abstraction.
<code>chunk_overlap</code> must be smaller than <code>chunk_size</code>	Invalid RAG configuration.	Lower <code>CHUNK_OVERLAP</code> or raise <code>CHUNK_SIZE</code> .
Conversation memory lost after restart	Current checkpointer is in-memory.	Use a durable LangGraph checkpointer for production conversations.

Release Checklist

1. Confirm target provider, model, and API key are configured in the target environment.
2. Run `uv sync --frozen`, `uv run pytest`, `uv run ruff check .`, and `uv run mypy`.
3. Confirm knowledge base documents are current and approved for indexing.
4. Run `uv run research-agent ingest --force` or restore a trusted vector store artifact.
5. Run a smoke test question that exercises calculator and knowledge base retrieval.
6. Verify production logs reach the log collector.
7. Confirm rollback plan: previous code version, previous knowledge base corpus, and previous vector store artifact.

Quick Command Reference

```
# install
uv sync

# configure
cp .env.example .env

# ingest documents
uv run research-agent ingest
uv run research-agent ingest --force

# run
uv run research-agent ask "Your question"
uv run research-agent chat
uv run research-agent ask "Your question" --thread my-thread

# validate
uv run pytest
uv run ruff check .
uv run ruff format .
uv run mypy
uv build

# production-style environment
export ENVIRONMENT=production
export LOG_LEVEL=INFO
export LLM_PROVIDER=groq
export GROQ_API_KEY="from-secret-manager"
```